

Question 1: Water Jug Problem

Question: Using a 4-gallon jug and a 3-gallon jug (no markings) and a pump, obtain exactly 2 gallons of water in the 4-gallon jug.

Step 1: Understanding the problem

- Two jugs: capacity 4 gallons (Jug X) and 3 gallons (Jug Y). Both are initially empty.
- Allowed operations: fill a jug from the pump, empty a jug on the ground, pour water from one jug to the other. No other measuring device is available.
- **Goal:** exactly 2 gallons in the 4-gallon jug.

Step 2: Problem formulation (state space)

- **State:** ordered pair (x, y) , where x = gallons in the 4-gallon jug ($0 \leq x \leq 4$) and y = gallons in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** $(0, 0)$.
- **Goal state:** $(2, n)$ for any n , i.e. $x = 2$.
- **Path cost:** each operation costs 1, so the best solution is the one with the fewest operations.

Step 3: Production rules (operators)

Rule	Condition (x, y)	Result	Action
1	$x < 4$	$(4, y)$	Fill the 4-gallon jug
2	$y < 3$	$(x, 3)$	Fill the 3-gallon jug
3	$x > 0$	$(0, y)$	Empty the 4-gallon jug on the ground
4	$y > 0$	$(x, 0)$	Empty the 3-gallon jug on the ground
5	$x + y \geq 4$ and $y > 0$	$(4, y - (4 - x))$	Pour from 3-gal jug into 4-gal jug until 4-gal jug is full
6	$x + y \geq 3$ and $x > 0$	$(x - (3 - y), 3)$	Pour from 4-gal jug into 3-gal jug until 3-gal jug is full
7	$x + y \leq 4$ and $y > 0$	$(x + y, 0)$	Pour all water from 3-gal jug into 4-gal jug
8	$x + y \leq 3$ and $x > 0$	$(0, x + y)$	Pour all water from 4-gal jug into 3-gal jug

Step 4: Solution (step by step)

Step	Rule	Action performed	x (4-gal)	y (3-gal)
0	–	Start: both jugs empty	0	0
1	2	Fill the 3-gallon jug from the pump	0	3
2	7	Pour all 3 gallons into the 4-gallon jug	3	0
3	2	Fill the 3-gallon jug again from the pump	3	3
4	5	Pour from the 3-gallon jug into the 4-gallon jug until it is full (only 1 gallon fits)	4	2
5	3	Empty the 4-gallon jug on the ground	0	2
6	7	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	2	0

State-space path: $(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$.

Step 5: Alternative solution (also 6 steps)

$(0,0) \rightarrow (4,0)$ [Rule 1] $\rightarrow (1,3)$ [Rule 6] $\rightarrow (1,0)$ [Rule 4] $\rightarrow (0,1)$ [Rule 7] $\rightarrow (4,1)$ [Rule 1] $\rightarrow (2,3)$ [Rule 6].

Step 6: Interpretation of result

After 6 operations the 4-gallon jug contains exactly 2 gallons, so the goal is achieved. Both solutions need the minimum of 6 operations, which is what a breadth-first search of the state space would also find. Verification: in the first solution, 6 gallons were pumped in $(3 + 3)$, 4 gallons were emptied on the ground in Step 5, and $6 - 4 = 2$ gallons remain, all in the 4-gallon jug. ✓

Question 2: Mars Rover – Intelligent Agent Analysis

Question: Analyse how a Mars Rover explores and analyses samples on Mars by answering (i) to (v).

PEAS description (summary)

PEAS	Mars Rover
Performance measure	Number and scientific value of samples analysed, distance explored safely, mission lifetime, energy efficiency, successful data transmission, no damage to the rover.
Environment	Martian surface: rocks, sand, slopes, craters, dust storms, extreme temperatures, thin atmosphere, day/night cycle.
Actuators	Wheels and steering motors, robotic arm, drill/scoop, sample-handling system, antenna, camera pan-tilt unit, heaters.
Sensors	Cameras, hazard-avoidance sensors, spectrometers, inertial/tilt sensors, wheel encoders, temperature, wind and radiation sensors, battery/health monitors.

(i) Percepts of the agent

- Images from navigation, panoramic and hazard-avoidance cameras (terrain, rocks, slopes).
- Range/obstacle data used to detect craters, boulders and cliffs.
- Position and motion: wheel odometry, inertial measurement unit, tilt and sun-sensor readings.
- Sample data: spectrometer and chemical/mineral analysis readings, drill contact and force feedback.
- Weather: temperature, atmospheric pressure, wind speed, dust level, radiation.
- Internal status: battery charge, solar-panel power, motor currents, component temperatures.
- Communication: commands and updates received from Earth (via orbiter relay), mission clock.

(ii) Characterisation of the operating environment

Property	Type	Reason
Observability	Partially observable	Sensors see only nearby terrain; buried rocks, soft sand and the far side of hills are unseen.
Determinism	Stochastic	Wheel slip, dust and wind make the result of an action uncertain.
Episodic / Sequential	Sequential	Current moves affect future position, energy and sample choices.

Property	Type	Reason
Static / Dynamic	Dynamic	Light, temperature and dust storms change while the rover is deliberating.
Discrete / Continuous	Continuous	Positions, speeds, angles and sensor values are continuous.
Agents	Single agent	Only the rover acts on the terrain; the Earth team gives commands.
Known / Unknown	Largely unknown	Terrain is not fully mapped; the agent must learn as it explores.

Additional constraint: one-way communication delay between Earth and Mars is several minutes (roughly 3–22 minutes), so the rover cannot be controlled in real time and must act autonomously.

(iii) Actions the agent can take

- Move forward/backward, steer, turn in place, stop.
- Plan a path and avoid obstacles; climb or avoid slopes.
- Capture images and pan/tilt the camera mast.
- Extend the robotic arm, brush/abrade a rock surface, drill or scoop a sample.
- Place instruments on the sample and run analysis; store sample in the cache.
- Transmit data to Earth and receive instructions.
- Manage power: orient solar panels, switch heaters on/off, sleep/recharge, enter safe mode on a fault.

(iv) Evaluating the performance of the agent

The performance measure judges the results in the environment, not the agent's internal behaviour. Criteria:

- Number of samples collected and analysed, and their scientific value.
- Distance and area covered safely in the mission time.
- Mission lifetime (survival) and damage/risk incidents (should be zero).
- Energy used per task and time taken per task.
- Accuracy of analysis and amount of useful data delivered to Earth.

An example scoring function: **Performance** = **w1(samples) + w2(science value) + w3(distance) – w4(damage) – w5(energy used)**, where w1...w5 are weights set by mission scientists.

(v) Most suitable agent architecture and why

Answer: a utility-based agent built on a model-based, goal-based design (with a learning element and a simple reflex safety layer).

- **Model-based:** the environment is partially observable, so the rover keeps an internal map, its position, battery level and sample status.
- **Goal-based:** it has explicit goals (reach a target site, collect a sample, return data) and plans action sequences to achieve them.
- **Utility-based:** many goals conflict (science value vs. energy vs. risk), so it picks the action with the highest expected utility.
- **Reflex layer:** immediate rules such as 'stop if tilt is too high' or 'enter safe mode if battery is low' guarantee quick safe responses.
- **Learning element:** improves terrain classification and navigation from experience.

A simple reflex agent is unsuitable (needs memory and planning) and pure remote control is impossible because of the communication delay, so an autonomous utility-based agent is the best choice.

Question 3: 8-Queens Problem – Formulation and Search

Question: Formulate the problem components and explore the search space for the 8-Queens problem.

Step 1: Understanding the problem

Place 8 queens on an 8×8 chessboard so that no two queens attack each other, i.e. no two share the same row, column or diagonal. The configuration in the figure is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal.

Step 2: Problem formulation (incremental formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board, one per column starting from the leftmost column, with no queen attacked. Total = 2,057 states.
Initial state	Empty board (no queens placed).
Actions	Add a queen to a square in the leftmost empty column such that it is not attacked by any queen already placed.
Transition model	Returns the board with the new queen added in the chosen square.
Goal test	8 queens are on the board and none is attacked.
Path cost	Not important – only the final arrangement matters (each placement may be taken as cost 1; every solution is at depth 8).

Step 3: Comparing formulations (size of search space)

Formulation	Idea	States
Complete-state (naive)	Place all 8 queens anywhere, one at a time ($64 \times 63 \times \dots \times 57$)	$\approx 1.8 \times 10^{14}$
Permutation	One queen per column and per row	$8! = 40,320$
Incremental (chosen)	Add queens column by column, only in non-attacked squares	2,057

The incremental formulation prunes illegal states early, so the search space shrinks drastically.

Step 4: Exploring the search space with uninformed strategies

Strategy	Behaviour on 8-Queens
Breadth-First Search	Complete, but all solutions lie at depth 8, so it must generate nearly the whole tree level by level; memory requirement is very large.

Strategy	Behaviour on 8-Queens
Depth-First Search (with backtracking)	Goes down to depth 8 quickly and stores only one path (memory $O(\text{depth}) = 8$). On a dead end it backtracks. Best suited to this problem.
Uniform-Cost Search	All steps cost 1, so it behaves like BFS – no advantage.
Iterative Deepening	Complete with low memory, but re-expands upper levels; unnecessary since the goal depth (8) is known.

Step 5: Depth-first search with backtracking – trace

Squares are written as (column, row). Rows are tried from 1 to 8.

Step	Action	Result
1	Column 1: try row 1	Q1 at (1,1) – accepted
2	Column 2: rows 1 and 2 are attacked (same row / diagonal); try row 3	Q2 at (2,3) – accepted
3	Column 3: rows 1–4 attacked; row 5 is safe	Q3 at (3,5) – accepted
4	Column 4: row 1 attacked; row 2 safe	Q4 at (4,2) – accepted
5	Column 5: rows 1–3 attacked; row 4 safe	Q5 at (5,4) – accepted
6	Column 6: every row 1–8 is attacked	Dead end – backtrack to column 5, then 4, 3, ...
7	Continue trying the next unused row at each level	Systematic backtracking explores other branches
8	First complete safe placement reached	Goal test satisfied

Step 6: Solution

Queens (column → row): **1→1, 2→5, 3→8, 4→6, 5→3, 6→7, 7→2, 8→4**, i.e. the row sequence (1, 5, 8, 6, 3, 7, 2, 4).

Verification: all rows are different, all columns are different, the values (row – column) are 0, 3, 5, 2, –2, 1, –5, –4 (all different) and the values (row + column) are 2, 7, 11, 10, 8, 13, 9, 12 (all different), so no two queens share a diagonal. ✓

Row \ Col	1	2	3	4	5	6	7	8
1	Q							
2							Q	
3					Q			

4								Q
5		Q						
6				Q				
7						Q		
8			Q					

Figure: One valid 8-Queens solution (Q = queen)

Step 7: Interpretation

The problem has 92 distinct solutions (12 fundamental solutions, excluding rotations and reflections). Backtracking DFS finds a solution efficiently because it rejects attacked squares immediately and needs memory for only one path of length 8.

Question 4: OLA Cab Booking – Problem-Solving Agent

Question: Identify the type of problem-solving agent for booking a cab (mini, micro, sedan, shared, prime, etc.) and write the pseudocode.

Step 1: Type of agent

A **goal-based problem-solving agent** is used. It has a clear goal (reach the destination) and works by formulating the goal, formulating the problem, searching for a sequence of actions (route and cab), and then executing it. Because the customer can choose among several cab types according to comfort, fare and waiting time, the cab choice is made with a **utility function** (utility-based element) that picks the best option.

Step 2: Problem formulation

Component	Definition
Percepts	Pickup location, drop location, available cab types, fare estimate, ETA, traffic, driver availability, customer preference.
Goal	Customer is at the destination location.
Initial state	Customer at the pickup location with the app open.
Actions	Enter locations, view cab types, select cab type, confirm booking, track the cab, board, ride, pay, rate.
Transition model	Each action changes the state: Pickup → Booked → Driver assigned → Cab arrived → On trip → At destination.
Goal test	Current location = destination.
Path cost / utility	Fare, travel time and waiting time (lower is better) and comfort (higher is better).

Step 3: Pseudocode – problem-solving agent

```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence (initially empty)
    state, the current description of the world
    goal, initially null
    problem, a problem formulation
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)           // reach destination
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)                  // route + cab type
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Step 4: Pseudocode – cab booking procedure

```
procedure BOOK-CAB(pickup, drop, preference)
  1. Read pickup and drop; if invalid then ask the user to re-enter
  2. cabList  $\leftarrow$  GET-AVAILABLE-CABS(pickup)    // Mini, Micro, Sedan, Share, Prime, ...
  3. if cabList is empty then display "No cabs available"; return
  4. for each cab c in cabList do
    fare(c)  $\leftarrow$  ESTIMATE-FARE(pickup, drop, c)
    eta(c)  $\leftarrow$  ESTIMATE-ARRIVAL-TIME(c, pickup)
     $U(c) \leftarrow w_1 \cdot \text{comfort}(c) - w_2 \cdot \text{fare}(c) - w_3 \cdot \text{eta}(c)$  // utility
  5. Display cab types with fare and ETA
  6. if the user selects a cab c' then chosen  $\leftarrow$  c'
    else chosen  $\leftarrow$  the cab with maximum U(c)
  7. route  $\leftarrow$  SEARCH-BEST-ROUTE(pickup, drop)    // e.g. A* / least time
  8. if user confirms the booking then
    driver  $\leftarrow$  ASSIGN-DRIVER(chosen)
    if driver rejects then go back to step 4 (re-assign / next best cab)
  9. TRACK(driver) until the cab reaches pickup
  10. Start the trip; follow route; update the state continuously
  11. while current location  $\neq$  drop do continue the trip
  12. End trip; compute final fare; PAY(fare)
  13. Ask the user for rating and feedback
  14. return success
```

Step 5: Interpretation

The agent first formulates the goal and problem, then searches for the best option and executes the plan while monitoring the state (replanning if the driver cancels or traffic changes). Utility-based cab selection balances comfort, fare and waiting time according to the customer's preference.

Question 5: Uniform Cost Search (UCS) on the Delivery Network

Question: Find the least-cost path from warehouse S to warehouse G using UCS.

Step 1: Understanding the problem

The network is a weighted directed graph. UCS always expands the frontier node with the smallest path cost $g(n)$ using a priority queue. Edge costs read from the graph:

Edge	Cost	Edge	Cost
$S \rightarrow A$	1	$B \rightarrow D$	3
$S \rightarrow G$	12	$C \rightarrow D$	1
$A \rightarrow B$	3	$C \rightarrow G$	2
$A \rightarrow C$	1	$D \rightarrow G$	3

Step 2: Method selection

UCS is complete and optimal when all edge costs are positive. It is the right choice because the aim is the minimum-cost route, not the route with the fewest edges. The goal test is applied when a node is **selected for expansion** (not when it is generated).

Step 3: Step-by-step execution

Frontier entries are written as Node(g), sorted by lowest g . Ties are broken alphabetically.

Step	Node expanded	Frontier after expansion	Remarks
0	–	S(0)	Start with initial node S
1	S(0)	A(1), G(12)	Generate $A = 0+1$, $G = 0+12$
2	A(1)	C(2), B(4), G(12)	Generate $B = 1+3 = 4$, $C = 1+1 = 2$
3	C(2)	D(3), B(4), G(4)	$D = 2+1 = 3$; $G = 2+2 = 4$ replaces G(12) (cheaper)
4	D(3)	B(4), G(4)	G via D = $3+3 = 6 > 4$, discarded
5	B(4)	G(4)	D via B = $4+3 = 7 > 3$, discarded
6	G(4)	–	Goal selected → stop

Order of expansion: $S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow G$.

Step 4: Path retrieval (using parent pointers)

$G \leftarrow C \leftarrow A \leftarrow S$, so the path is $S \rightarrow A \rightarrow C \rightarrow G$.

Total cost = $1 + 1 + 2 = 4$

Step 5: Verification with all possible paths

Path	Cost	Remark
$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 1 + 2 = 4$	Minimum (optimal)
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3 = 6$	Higher
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1 + 3 + 3 + 3 = 10$	Higher
$S \rightarrow G$	12	Highest

Step 6: Interpretation of result

The least-cost delivery route is $S \rightarrow A \rightarrow C \rightarrow G$ with a total travel cost of **4 units** (fuel + time). The direct route $S \rightarrow G$ costs 12 units, so the UCS route saves 8 units (about 67 %). The company should route its packages through warehouses A and C. Since UCS is optimal for positive costs, no cheaper route exists.